CITS3007 Secure Coding – Group project 2026, phase 2 brief

Version: 1.0

Due	11:59 pm, Thursday 21 May (week 12)
Weight	15 marks (15% of final grade)
Groups	Same groups as for phase 1

1. Background

In phase 1, your group implemented a parser for the BUN (**B**inary **UN**ified assets) format in C11. In phase 2, you will take on the role of adversarial testers: your goal is to probe another group's BUN parser for security flaws.

Each group has been assigned two other groups' phase 1 submissions via email. You may choose *one* of the two assigned codebases to test. Do not test both.

The focus of phase 2 is *reproducible* findings. A flaw is only useful if someone else can confirm it – so everything your group discovers must be demonstrable by running the target parser in its normal execution environment (the CITS3007 SDE – see below), without modifying its source code.

2. Project rules

Rules for participation, peer evaluations, late submission, and academic conduct are as for phase 1.

You may use genAI tools to assist in coming up with ideas or generating test inputs, but all content must be reviewed by your group and you must note where you have used such tools.

You may not share your findings with other groups, nor discuss specific vulnerabilities in the target codebases, before the submission deadline.

See the CITS3007 website FAQ, Academic conduct and source citation for general expectations regarding source citation.

3. Deliverables

Phase 2 has two deliverables: a **reproduction package** and a **findings report**.

Both must be submitted via the Moodle LMS by the due date above, as two files: a `.zip` file, and a PDF report.

Only 1 group member may make a submission.

The `.zip` must contain your reproduction package (see Task 2); the report should discuss your findings (see Task 3).

Ensure that your group number is clearly displayed on the first page or coversheet of every deliverable. List the name, student number, and GitHub (or other VC hosting service) username of each group member.

a. Availability for follow-up questions

Group members should make themselves available during the study week (the week following week 12) in case their lab facilitator has questions about the submission. This is not a scheduled event – facilitators will contact groups only if needed.

b. Code expectations

See the CITS3007 website FAQ, "'Long answer' questions requiring code" for general expectations regarding code quality.

c. Report expectations

See the CITS3007 website FAQ, "What are the formatting expectations for project reports?". PDF reports only.

4. Task 1: Setup

This task is not directly assessed, but is essential preparation.

Before beginning your testing, your group should:

- Identify which of the two assigned codebases you will test, and record that decision.
- Confirm that the target codebase builds and runs on the CITS3007 standard development environment (SDE): Ubuntu 20.04, x86-64.
- Familiarise yourselves with the target code, its `Makefile`, and any `setup.sh` it provides.
- Set up your own private repository for phase 2 work.
  - You can use the same repository from your phase 1 work if you wish, but we recommend creating a new "orphan" branch in which to work on phase 2. An "orphan" branch is one that shares no common history with your main or master branch.
  - NB: Ensure that your repository is **private** – when markers need access to it, they will ask to be added as contributors (or will supply a "deploy key" for you to add which lets them view the repository, with instructions on how to add it).
- We recommend you do *not* use MS Word to write the report. Instead, use a plain text format (such as Markdown, LaTeX, RTF, or similar). See Collaborating on your report for tips on writing a group report, and on converting from Markdown to PDF.

5. Task 2: Reproduction Package

Your group must submit a reproduction package: a directory tree containing everything needed to demonstrate each flaw you have found.

5.1. Structure and submission

The reproduction package must be packaged as a `.zip` file, and should contain:

- A `Makefile` with a `reproduce` target. Running `make reproduce` must execute all of your test cases against the target parser and produce output clearly indicating which tests trigger a flaw and which do not.
- Any BUN files (valid, malformed, or otherwise crafted) required to trigger the flaws.

  We recommend using directories like `tests/fixtures/valid` and `tests/fixtures/invalid` to store these in, but the exact structure is up to you.

  - You may use helper scripts to generate BUN files if you wish, instead of adding BUN files to your `.zip` – this is entirely up to you.
- Any helper scripts used to automate or support reproduction.
- Do not include the target group's source code or compiled binaries in your submission – we will supply that ourselves when marking.

  - Instead, your package should assume the person running the tests has unzipped the group's source into a directory `target` within your reproduction package.

Your tests will probably need to *build* the target binary with different compilation flags – a simple way to do that is with a Makefile recipe like:

```
CUSTOM_CFLAGS = -std=c11 -fsanitize=address -fno-omit-frame-pointer -g -O2

rebuild_with_sanitizers:
  # build bun_parser under target
  cd target && make CFLAGS="$(CUSTOM_CFLAGS)"
  # rename for later use
  mv target/bun_parser bun_parser_with_sanitizers
```

But the exact approach is up to you.

5.2. Rules of engagement

The following rules govern how you may compile and run the target parser.

Compilation flags

You may add or remove compilation flags when building the target parser, subject to these constraints:

- You **may** add optimisation flags (e.g. `-O2`, `-O3`).
- You **may** enable sanitizers (e.g. `-fsanitize=address`, `-fsanitize=undefined`).
- You **may** add or remove debugging symbols (e.g. `-g`, `-g0`).
- You **may not** remove the `-std=c11` flag. Target groups are entitled to assume their code is compiled as C11.
- You **may not** add or remove preprocessor symbol definitions (e.g. `-DSOME_SYMBOL`, `-USOME_SYMBOL`). This would be equivalent to altering the source code.

Test execution environment

All findings must be reproducible on the CITS3007 SDE (Ubuntu 20.04, x86-64). The target group was entitled to assume their code would be run in this environment, so any test that relies on behaviour specific to a different OS or architecture is not eligible.

You may use a Docker container to control the execution environment (for instance, to impose memory limits). The Docker container itself should be able to be run on the SDE (e.g. via Docker on Ubuntu 20.04) – you can install Docker within the CITS3007 SDE by running `sudo apt install docker.io`.

If you do so, document this clearly in your report, and provide instructions for running your tests within a Docker container. Any memory limit you impose must allow the process to use at least 1 GB of RAM.

Source code

Your tests may not modify the target group's source code. Your findings must be reproducible by invoking the parser normally via its `main` entry point. (You can *temporarily* modify the source code for exploration purposes, e.g. while invoking fuzzers or similar tools – but your final deliverable in this case should be a BUN file which triggers a failure when run using the normal `bun_parser` executable.)

5.3. What counts as a finding

We are primarily interested in three categories of failure:

Category	Description
Crash / abnormal exit	The parser terminates via a signal (e.g., segfault, abort) or via a sanitizer-detected error, rather than exiting cleanly with a BUN status code.
Excessive memory use	The parser reads from or writes to more than 1 GB of memory during execution. (Recall that the spec asks for sub-linear memory usage – the parser should seek and iterate, not load files wholesale into RAM.)
Hang	The parser produces no output to stdout/stderr and no process termination for more than 5 seconds.

Incorrect output (wrong field values, wrong exit codes, missed or spurious violation reports) may also be reported, and up to 2 bonus marks (in total, not per finding) may be awarded at the marker's discretion for well-evidenced incorrect-output findings. These bonus marks can offset losses elsewhere but cannot take a group's total above 15.

If your finding relies on a specific interpretation of an ambiguous part of the BUN specification, state that interpretation as an assumption in your report. Reasonable assumptions will be credited.

A finding is only required to demonstrate a flaw in the mandatory parts of the BUN specification. A report is considered thorough if it covers all mandatory behaviours. Optional extensions (such as zlib compression) need not be tested, though you may do so if you wish.

A finding must result in observable incorrect behaviour (i.e. crash, sanitizer error, incorrect output, hang, or excessive resource usage). Issues that cannot be demonstrated to affect observable behaviour aren't eligible for inclusion.

6. Task 3: Findings Report

Your report must be structured as follows.

6.1. Section F1: Introduction

Briefly state:

– Which codebase you chose to test (group number), and why you chose it over the alternative.
– Any general observations about the codebase (e.g. structure, apparent coding practices) that are relevant to your testing approach.

6.2. Section F2: Assumptions and method

Describe your general testing approach:

– Any general assumptions you made (e.g. about the build environment, compilation flags used, or interpretation of the spec).
– What techniques and tools you used – for instance: manual crafting of test inputs, fuzzing, dynamic analysis tools (Valgrind, sanitizers), static analysis, or debugging. Note that tools are a legitimate part of your *process*, but the final deliverable must be a replicable test that triggers failure through normal parser execution.
– How you systematically covered the mandatory parts of the specification.

6.3. Section F3: Findings

For each finding, provide the following:

Field	Content
ID	A short identifier, e.g. F-01, F-02.
Category	Crash, excessive memory use, hang, or incorrect output.
Description	A clear, concise description of the flaw.
Spec reference	The section(s) of the BUN specification that are violated, if applicable.
Assumptions	Any interpretations of the spec required for this finding to constitute a flaw. State these explicitly.
Reproduction	Step-by-step instructions to reproduce the finding, including the exact command(s) to run, any relevant BUN input file(s), and compilation flags used. A marker must be able to follow these steps on the SDE and observe the failure.
Expected behaviour	What a correct parser should do.
Actual behaviour	What the target parser actually does.

6.4. Section F4: Conclusion

Summarise your findings. If you found no flaws, present a clear and well-argued case that the codebase correctly handles the mandatory parts of the spec – your test cases should demonstrate that each expected behaviour was verified and matched. A thorough "clean bill of health" supported by comprehensive tests can receive full marks.

If you did find flaws, briefly characterise the nature of the issues found (e.g. are they clustered around a particular part of the spec, or a particular type of failure?).

You may include any recommendations to the target group, though this is not required.

7. Marking

Phase 2 is marked out of **15** and contributes **15%** of your final grade.

The marking scheme is designed so that groups are not penalised for receiving a codebase that happens to be very robust – a thorough and well-documented investigation that concludes "no flaws found" can receive full marks.

Component	**Marks**
`Makefile` correctness: `make reproduce` runs all tests and reports outcomes correctly	3
Reproduction package quality: well-organised, reproducible, automated tests	5
Report quality: clear, consistent, explicit, systematic report	7
Total	**15**
Bonus: incorrect-output findings (marker's discretion)	+2

Note: bonus marks may offset losses elsewhere but cannot take a group's total above 15.

Reproduction package and report are assessed as being proficient, satisfactory, or not yet proficient, using the criteria outlined below.

Reproduction package quality

Proficient (4–5 marks):

- All findings from report are reproducible
- Tests are well organised and clearly mapped to findings
- Minimal manual intervention required
- Robust to small environmental differences

Satisfactory (2–3 marks):

- Most findings reproducible, but some friction or ambiguity
- Organisation adequate but not always clear
- Some manual steps or assumptions required

Not yet satisfactory (0–1 marks):

- Reproduction unreliable or incomplete
- Significant ambiguity or missing steps
- Markers cannot reliably verify findings

Findings report quality

Proficient (6–7 marks):

- All or nearly all findings are clearly described and well justified
- Reproduction steps align with the package
- Assumptions are explicit and reasonable
- Coverage of mandatory behaviours is systematic or well-argued

Satisfactory (4–5 marks):

- Findings mostly clear, with some gaps or imprecision
- Reproduction generally understandable but occasionally unclear
- Some assumptions implicit or weakly justified
- Coverage present but not obviously systematic

Not yet satisfactory (0–3 marks):

- Findings unclear, poorly justified, or inconsistent
- Reproduction steps incomplete or ambiguous
- Weak or missing connection to specification
- Little evidence of systematic testing

8. Advice and tips

8.1. Getting help

Lab facilitators can provide guidance throughout the project. Facilitators cannot provide solutions or hints about specific vulnerabilities, but can help you think about your testing approach. Discuss your progress during labs.

8.2. Testing approach

A good testing strategy covers the full space of mandatory spec behaviours systematically, rather than trying a small number of inputs and hoping for a crash. Some areas to consider:

- Boundary values: zero-length sections, maximum field values, offsets pointing to the last byte of a section.
- Overlapping sections, sections that extend beyond the end of the file.
- Compressed data with mismatched `uncompressed_size`, or RLE data with zero `count` values.
- Asset names at the edge of or outside the string table bounds.
- Files with `asset_count` of zero, or very large `asset_count` values.
- Integer arithmetic: consider what happens when offsets and sizes, treated as 64-bit values, are near the limits of their type.

8.3. Tools

You are encouraged to use any tools that help you identify potential flaws – fuzzers, Valgrind, GDB, static analysers, sanitizers, and so on. Document their use in your methods section. Remember that the final deliverable must be a replicable test via normal parser execution, not a tool-specific artifact.

8.4. Group organisation

Consider assigning roles such as:

- **Test development** – crafting and automating the test inputs
- **Analysis** – reading the target code and spec to identify likely weak points
- **Report writing** – drafting and reviewing the findings report
- **Reproduction verification** – confirming that each finding is cleanly reproducible from scratch on the SDE

These are suggestions – what matters is that all members contribute meaningfully and can speak to the work if asked.

8.5. Collaborating on your report

We recommend using the same tools you use for code – Git, Make, and plain-text formats – for your report as well. These are tried and tested for team collaboration: incorporating a teammate's changes is just a `git merge`, generating a PDF is just `make all`, and it's straightforward at the end to see who contributed what.

Our recommendation is to use the Markdown format to write your report. To produce a PDF from Markdown, we recommend installing Pandoc and Typst. Together they can produce professional-

quality PDFs on Windows, Linux, and macOS. For Linux, we provide a skeleton `group-XX-phase-2-report.md` and a `Makefile` to build it, along with a `setup.sh` that installs both tools from GitHub.

If you prefer not to use Typst, other options for generating a PDF from Markdown include:

- Using Pandoc to convert your Markdown to HTML, then using your browser's "print to PDF" functionality
- Using Pandoc to convert to MS Word or another word processor, then exporting to PDF from there
- Install the Markdown PDF extension in VS Code, and export PDFs from within your editor
- Install a cross-platform tool like mdPDFinator